Operating Systems Lab Manual

B.Tech. CSE (II YEAR – III/IV SEM)

(2024-2025)

DEPARTMENT OF COMPUTER ENGINEERING & APPLICATIONS

Institute of Engineering & Technology



GLA University

17km Stone, NH-19, Mathura-Delhi Road P.O. Chaumuhan,
Mathura - 281406 (Uttar Pradesh) India

LINUX COMMANDS

GLA University, Mathura

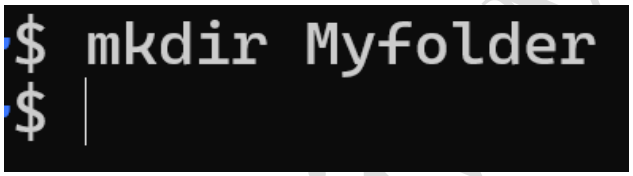
Prepared by: Dr. Premnarayan Arya

1. Implement the following basic Linux commands in Ubuntu/Terminal

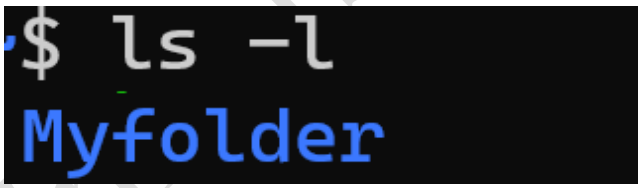
mkdir	ls	cd	cat	touch	man
pwd	who	whoami	date	cal	rm
rmdir	head	tail	more	less	cp
mv	echo	history	sed	grep	pipe
cut	uniq	chmod	du	diff	last

Solution 1:

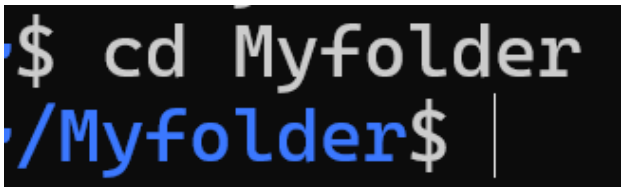
- **mkdir:** create a new directories or folder.



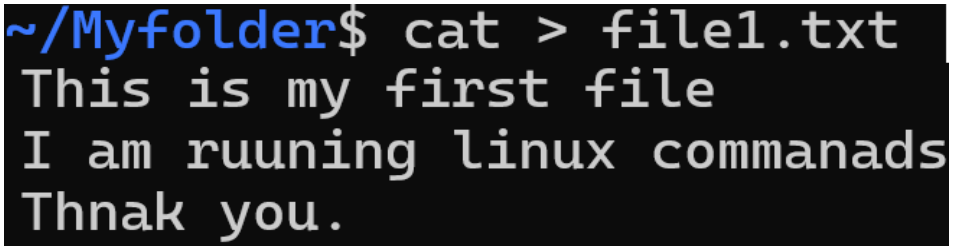
- **ls:** To display all files and directories.



- **cd:** To change a directory.



- **cat:** To create a new file with contents.



GLA University Mathura

- **touch:** To create an empty (or blank) file, also create multiple files.

```
/Myfolder$ touch file2.txt
/Myfolder$ touch file3.txt file4.txt file5.doc
```

- **man:** To display manual of any command.

```
~/Myfolder$ man cat
~/Myfolder$ man touch
```

- **pwd:** To display present working directory.

```
~/Myfolder$ pwd
/home/drpremnarya/Myfolder
```

- **who:** To display information about the users currently logged into the system. It provides details such as the username, terminal, login time, and sometimes the originating host.

```
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ who
drpremnarya pts/1        2024-09-05 12:48
```

- **whoami:** To display the effective username of the current user. It shows the username of the user who is currently logged in and executing the command.

```
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ whoami
drpremnarya
```

GLA University, Mathura

- **date:** print or set the system date and time.

```
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ date
Thu Sep  5 13:21:39 IST 2024
```

- **cal:** displays a calendar of the current month.

```
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ cal
    September 2024
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
```

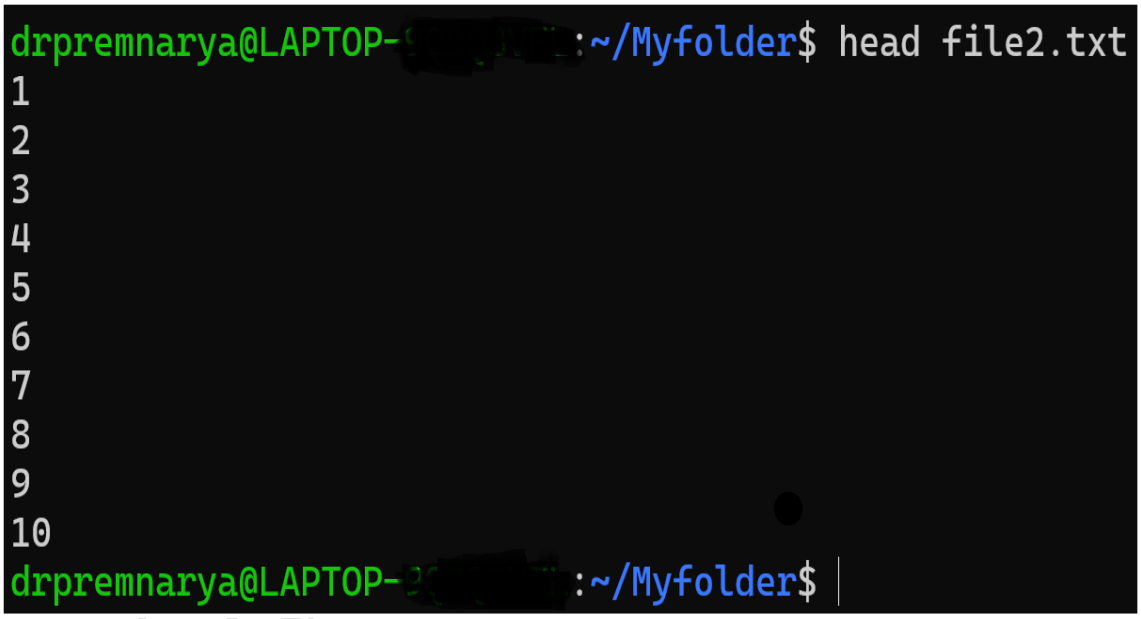
- **rm:** To remove a file.

```
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ ls
file1.txt  file2.txt  file3.txt  file4.txt  file5.doc
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ rm file1.txt
drpremnarya@LAPTOP-[redacted]:~/Myfolder$ ls
file2.txt  file3.txt  file4.txt  file5.doc
drpremnarya@LAPTOP-[redacted]:~/Myfolder$
```

- **rmdir:** To remove a directory/folder.



- **head:** To print 10 lines from top of any file.



- **tail:** To print 10 lines from bottom of any file.



- **more:** The more command in Linux is a simple text pager used to view the contents of a file one screen at a time. It allows users to navigate through large files or command output in a controlled manner. Unlike cat, which displays the entire content at once, more pauses after each screenful, making it easier to read.



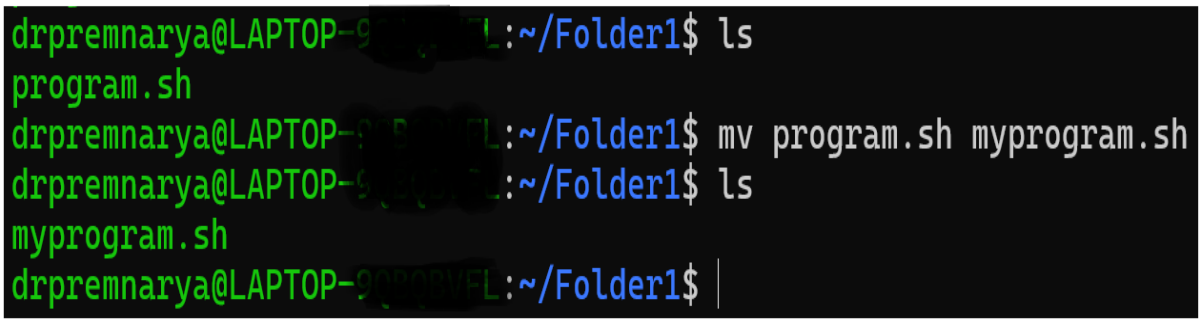
- **less:** The less command in Linux is a powerful text pager that allows users to view the contents of files one screen at a time, similar to the more command, but with more advanced features. Unlike more, less enables backward and forward navigation, as well as efficient searching and navigation within files.



- **cp:** The cp command in Linux is used to copy files and directories from one location to another. It is one of the most commonly used commands for managing files.



- **mv:** The mv command in Linux is used to move or rename files and directories. It is a versatile command that can be used both to relocate files to a different directory and to rename files and directories within the same location.
  - Rename program.sh file by new name myprogram.sh



  - Move/cut myprogram.sh file into the Folder2



- **echo:** The echo command in Linux is used to display a line of text or a string on the terminal. It is commonly used in scripts, batch files, and command-line operations to output text, display variables, and generate formatted text.

```
drpremnarya@LAPTOP-9QBQBVFL:~$ echo "welcome to You"
welcome to You
```

- **history:** The history command in Linux is used to display a list of previously executed commands in the terminal. It is a helpful tool for recalling past commands, rerunning them, or even editing them. The history command is particularly useful for tracking command usage, debugging, and learning.

```
drpremnarya@LAPTOP-9QBQBVFL:~$ history
  929  ls
  930  vim add_two_numbers_using_function.sh
  931  ls
  932  vim array_element2.sh
  933  vim array_element_nestedforloop.sh
  934  ls
  935  vim array_element_sum.sh
  936  cd script
  937  ls
  938  vim array_element2.sh
  939  ./array_element2.sh
  940  vim array_element2.sh
  941  ./array_element2.sh
  942  vim array_element2.sh
  943  ./array_element2.sh
  944  vim array_element2.sh
```

- **sed:** The sed command in Linux stands for "stream editor" and is used to perform basic text transformations and manipulations on files or input streams. It processes text line by line and is commonly used for tasks such as searching, find-and-replace operations, and text formatting.

```
drpremnarya@LAPTOP-[redacted]:~$ cat file1.txt
Hello
Hello
How are you
drpremnarya@LAPTOP-[redacted]:~$ sed 's/Hello/Hi/' file1.txt
Hi
Hi
How are you
drpremnarya@LAPTOP-[redacted]:~$
```

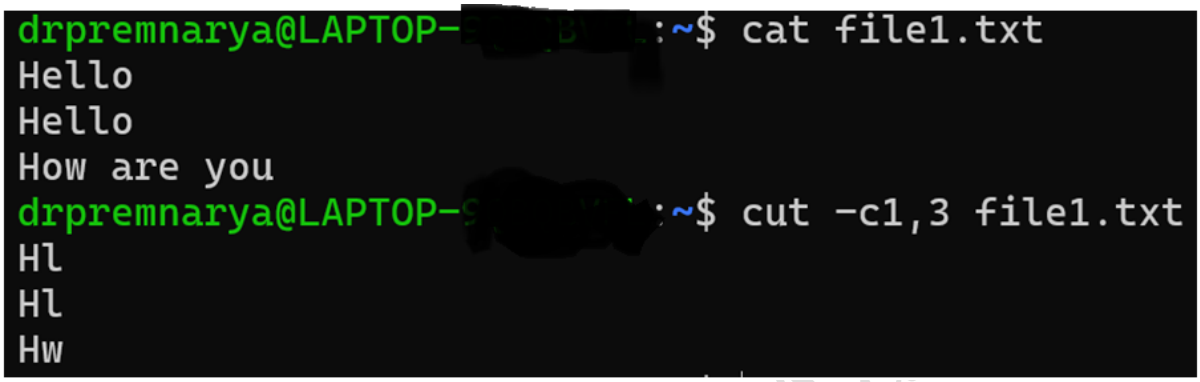
- **grep:** The grep command in Linux is used for searching text using patterns. It stands for "Global Regular Expression Print" and is commonly used to search through files and output lines that match a given pattern.

```
drpremnarya@LAPTOP-[redacted]:~$ cat file1.txt
Hello
Hello
How are you
drpremnarya@LAPTOP-[redacted]:~$ grep Hello file1.txt
Hello
Hello
```

- **pipe:** In Linux, the pipe command (|) is used to pass the output of one command as the input to another command. This allows you to chain commands together and perform complex data processing in a single line of code.

```
drpremnarya@LAPTOP-[redacted]:~$ echo "DrArya" | cut -c3
A
drpremnarya@LAPTOP-[redacted]:~$ echo "DrArya" | cut -c1
D
```

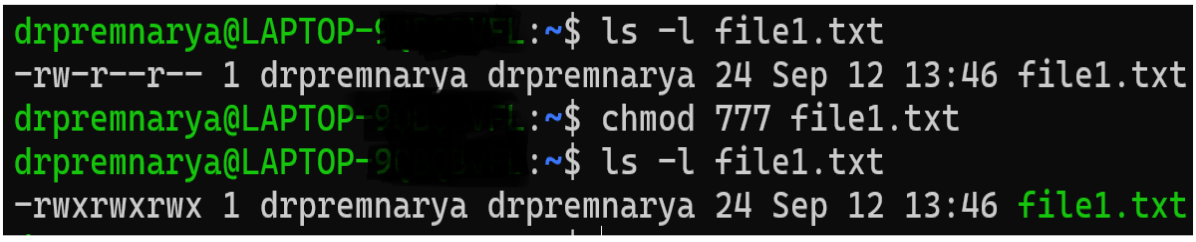
- **cut:** The cut command in Linux is used to extract sections from each line of input, such as files or standard input. It is commonly used for extracting columns or fields from structured text files or output from other commands.



- **uniq:** The uniq command in Linux is used to filter out or report repeated lines in a file or input stream. It is often used to remove duplicate lines or count occurrences of unique lines. The uniq command works best when used in combination with other commands, such as sort, to ensure that duplicate lines are adjacent.



- **chmod:** The chmod command in Linux is used to change the permissions of files and directories. The name chmod stands for "change mode," and it allows users to define who can read, write, or execute a file.



- **du:** The du (disk usage) command in Linux is used to estimate and display the amount of disk space used by files and directories. It's useful for checking disk space usage and managing storage.

```
drpremnarya@LAPTOP-[redacted]:~$ du
124     ./script1
8       ./Folder1/Folder2
12      ./Folder1
4       ./.cache
4       ./.config/procps
8       ./.config
4       ./.local/share/nano
8       ./.local/share
12      ./.local
4       ./snap/tree/common/.local/lib/locale
8       ./snap/tree/common/.local/lib
12      ./snap/tree/common/.local
16      ./snap/tree/common
20      ./snap/tree
24      ./snap
64      ./systemcalls
388     .
```

- **diff:** The diff command in Linux is used to compare the contents of two files line by line. It displays the differences between the files, making it useful for identifying changes, updates, or discrepancies.

```
drpremnarya@LAPTOP-[redacted]:~$ diff file1.txt program.sh
1,3c1,7
< Hello
< Hello
< How are you
---
> echo "Welcome to You in GLAU"
> echo "Ram"
> echo "OSLAB"
>
> echo "Enter your name"
> read a
> echo $a
```

- **last:** The last command in Linux is used to display a list of the most recent logins to the system. It reads from the /var/log/wtmp file, which records login, logout, and system boot events. This command is useful for auditing and monitoring user activity.

```
drpremnarya@LAPTOP-[redacted]L:~$ last
drpremna pts/1                        Thu Sep 12 12:55    gone - no logout
reboot   system boot  5.15.153.1-micro Thu Sep 12 12:55   still running
drpremna pts/1                        Tue Sep 10 10:21 - crash (2+02:34)
reboot   system boot  5.15.153.1-micro Tue Sep 10 10:20   still running
drpremna pts/1                        Mon Sep  9 10:08 - crash (1+00:12)
reboot   system boot  5.15.153.1-micro Mon Sep  9 10:08   still running
drpremna pts/1                        Sun Sep  8 20:29 - crash  (13:38)
reboot   system boot  5.15.153.1-micro Sun Sep  8 20:29   still running
drpremna pts/1                        Thu Sep  5 12:48 - crash (3+07:41)
reboot   system boot  5.15.153.1-micro Thu Sep  5 12:48   still running
drpremna pts/1                        Tue Sep  3 10:27 - crash (2+02:21)
reboot   system boot  5.15.153.1-micro Tue Sep  3 10:27   still running
drpremna pts/1                        Tue Aug 20 10:23 - crash (14+00:03)
reboot   system boot  5.15.153.1-micro Tue Aug 20 10:23   still running
drpremna pts/1                        Tue Aug 13 12:40 - crash (6+21:42)
reboot   system boot  5.15.153.1-micro Tue Aug 13 12:40   still running
```

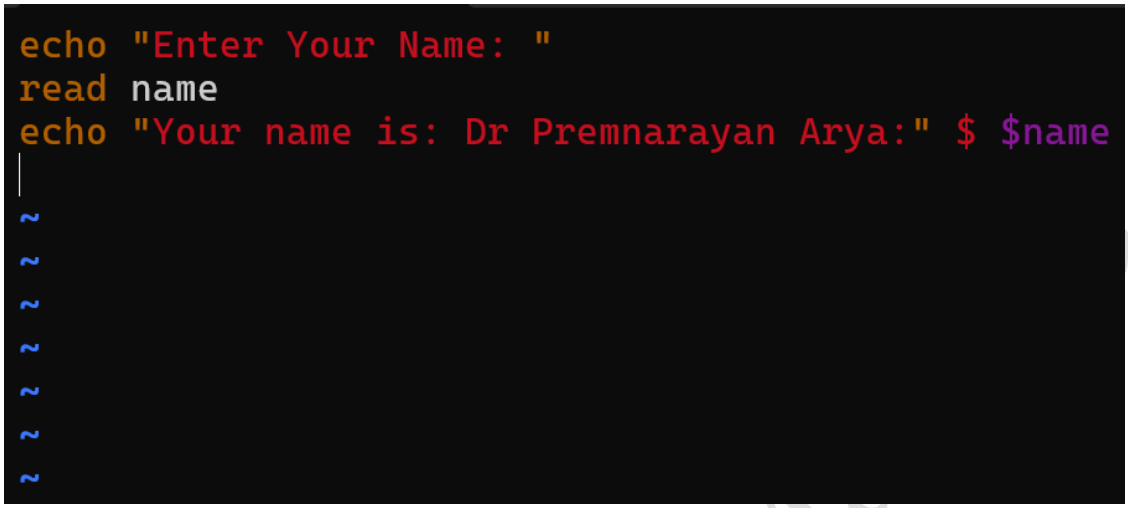
GLA University

SHELL SCRIPT PROGRAMMING

GLA University, Mathura

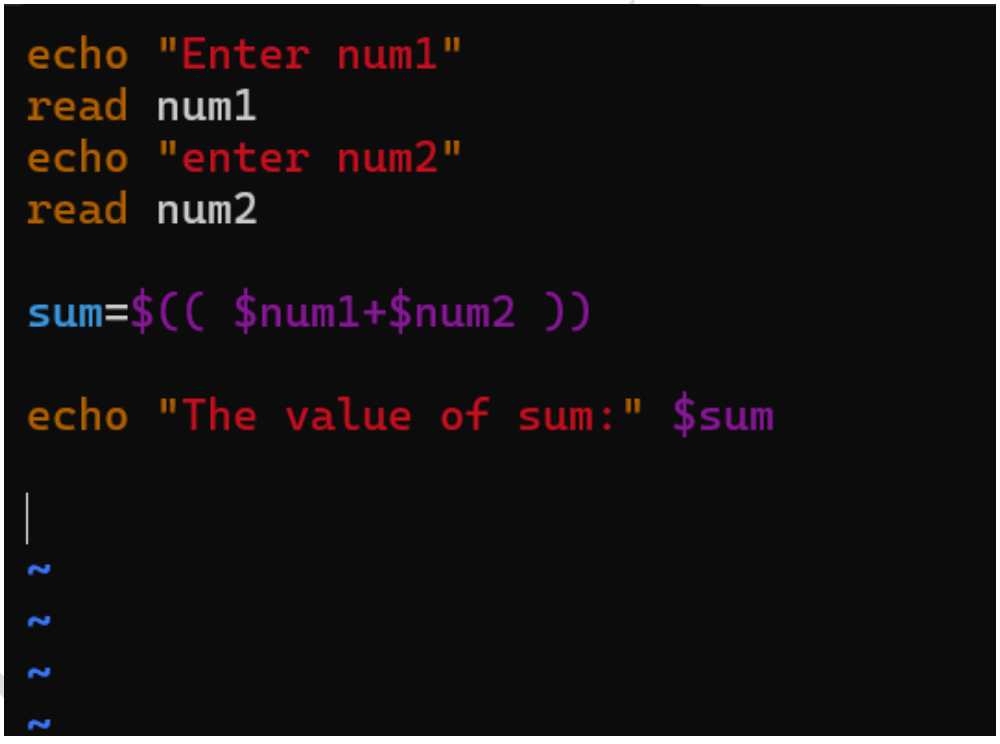
Prepared by: Dr. Premnarayan Arya

1. **write shell script programs to print name which input by the user.**



```
echo "Enter Your Name: "
read name
echo "Your name is: Dr Premnarayan Arya:" $ $name
```

2. **write shell script programs to sum of two numbers.**



```
echo "Enter num1"
read num1
echo "enter num2"
read num2

sum=$(( $num1+$num2 ))

echo "The value of sum:" $sum
```

3. **write shell script programs to print greater number among two numbers.**

```
echo "A shell script to find out the large between two number."

echo -n "Enter num1 number: "
read num1
echo -n "Enter num2 number: "
read num2

#if test $num1 -gt $num2
if [ $num1 -gt $num2 ]

then
        echo $num1 is greater than $num2.
else
        echo $num2 is greater than $num1.
fi
```

4. **write shell script programs to check whether a number is positive or negative.**

```
echo "Shell script to check whether a number is positive or negative"

echo -n "Enter a Number: "
read num

if [ $num -lt 0 ]
then
    echo "Negative"
elif [ $num -gt 0 ]
then
    echo "Positive"
else
    echo "Neither Positive Nor Negative"
fi
```

5. write shell script programs to check entered number is odd or even.

```
echo "Enter the Number"
read n
#rem=$(($n % 2))
#if [ $rem -eq 0 ]
if [ $(($n % 2)) -eq 0 ]
then
        echo "$n number is Even number"
else
        echo "$n number is Odd number"
fi
```

6. write shell script programs to sum of all digits enter by the user.

```
#Sum of all digits - Shell Script

echo "Enter a number"
read num

sum=0

while [ $num -gt 0 ]
do
    mod=$((num % 10))    #It will split each digits
    sum=$((sum + mod))   #Add each digit to sum
    num=$((num / 10))    #divide num by 10.
done

echo $sum
```

7. **write shell script programs to print numbers from 1 to 100 using for loop**

```
#Shell script to print numbers 1 to 100 using for loop

for((i=1;i<=100;i++))
do
    echo $i
done
~
~
~
```

8. **write shell script programs to print numbers from 1 to 10 using while loop.**

```
#shell script to print numbers 1 to 10

i=1
while [ $i -le 10 ]
do
    echo $i
    i=$(($i+1))
done

~
~
```

9. **write shell script programs to print factorial of a given number using for loop.**

```
#shell script to print factorial of a given number using for loop

echo "Enter a number"
read num

fact=1

for((i=2;i<=num;i++))
{
  fact=$((fact * i))  #fact = fact * i
}

echo $fact
```

10. **write shell script programs to print factorial of a given number using while loop.**

```
#shell script to factorial of a given number using while loop

echo "Enter a number"
read num

fact=1

while [ $num -gt 1 ]
do
  fact=$((fact * num))  #fact = fact * num
  num=$((num - 1))      #num = num - 1
done

echo $fact
```

11. write shell script programs to print elements using array.

```
#To print elements using array
fruits=(apple 'banana' 'cherry')
for i in "${fruits[@]}"
do
   echo I like $i
done


~
```

12. write shell script programs to print two array elements using nested for loop.

```
#To print two array elements using nested for loop

fruits=('apple' 'banana' 'cherry')
colors=('red' 'yellow' 'red')

for i in "${!fruits[@]}"
do
        echo The ${fruits[$i]} is ${colors[$i]}
done
```

13.write shell script programs to print sum of array elements.

```
#To print sum of array elements

arr=(10 20 30 40 50)

sum=0

for i in ${arr[@]}
do
        sum=$(($sum + $i)) #`expr $sum + $i`
done

echo $sum
```

14.write shell script programs to compare two strings are equal or not equal.

```
#shell script to compare two strings

read -p "Enter first strings: " str1
read -p "Enter second string: " str2

if [ $str1 == $str2 ]
then
    echo "equal"
else
    echo "not equal"
fi
```

CPU SHEDULING ALGORITHMS PROGRAMMING In SHELL SCRIPT LANGUAGE

GLA University, Mathura

Prepared by: Dr. Premnarayan Arya

1. **Implement FCFS CPU Scheduling Algorithm using Shell Script language to determine the average waiting time and average turnaround time given n processes and their burst times.**

Code:

```
# Bash script to implement first come first served CPU scheduling algorithm.

sort(){
        for ((i = 0; i<$n; i++))
        do

          for((j = 0; j<`expr $n - $i - 1`; j++))
          do

               if [ ${arrival_time[j]} -gt ${arrival_time[$((j+1))]} ]
               then
                  # swap
                  temp=${arrival_time[j]}
                  arrival_time[$j]=${arrival_time[$((j+1))]}
                  arrival_time[$((j+1))]=$temp
                  temp=${burst_time[j]}
                  burst_time[$j]=${burst_time[$((j+1))]}
                  burst_time[$((j+1))]=$temp
                  temp=${pid[j]}
                  pid[$j]=${pid[$((j+1))]}
                  pid[$((j+1))]=$temp
               elif [ ${arrival_time[j]} -eq ${arrival_time[$((j+1))]} ]
               then
                       if [ ${pid[j]} -eq ${pid[$((j+1))]} ]
                       then
                          temp=${arrival_time[j]}
```

GLA University, Mathura

```
                        arrival_time[$j]=${arrival_time[$((j+1))]}
                        arrival_time[$((j+1))]=$temp
                        temp=${burst_time[j]}
                        burst_time[$j]=${burst_time[$((j+1))]}
                        burst_time[$((j+1))]=$temp
                        temp=${pid[j]}
                        pid[$j]=${pid[$((j+1))]}
                        pid[$((j+1))]=$temp
                    fi
                fi
            done
        done
}

border(){
        z=121
        for ((i=0; i<$z; i++))
        do
        echo -n "-"
        done
        echo ""
}

findWaitingTime(){
        service_time[0]=0
        waiting_time[0]=0
        for ((i=1; i<$n; i++))
        do
                z=1
                y=`expr $i - $z`
                service_time[$i]=`expr ${service_time[$y]} + ${burst_time[$y]} `
                waiting_time[$i]=`expr ${service_time[$i]} - ${arrival_time[$i]}`
                if [ ${waiting_time[$i]} -lt 0 ]
                then
```

GLA University, Mathura

```
                waiting_time[$i]=0
        fi
    done
}

findTurnAroundTime(){
    for ((i=0; i<$n; i++))
    do
        tat[$i]=`expr ${waiting_time[$i]} + ${burst_time[$i]}`
    done
}

findAverageTime(){
    sort
    findWaitingTime
    findTurnAroundTime
    total_wt=0
    total_tat=0
    border
    printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" "Process Id" "Burst time"
"Arrival time" "Waiting time" "Turn around time" "Completion time"
    border
    for ((i=0; i<$n; i++))
    do
        total_wt=`expr $total_wt + ${waiting_time[$i]}`
        total_tat=`expr ${tat[$i]} + $total_tat`
        completion_time=`expr ${arrival_time[$i]} + ${tat[$i]}`
        printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" ${pid[$i]}
${burst_time[$i]} ${arrival_time[$i]} ${waiting_time[$i]} ${tat[$i]} $completion_time
        #echo "${burst_time[$i]}     ${arrival_time[$i]}     ${waiting_time[$i]}
${tat[$i]}        $completion_time"
    done
    border
    #avgwt=`echo "scale=3; $total_wt / $n" | bc`
```

GLA University, Mathura

```
        echo -n "Average waiting time ="
    printf %.3f\\n "$(($total_wt / $n))"
        #avgtat=`echo "scale=3; $total_tat / $n" | bc`
        echo -n "Average turn around time ="
    printf %.3f\\n "$(($total_tat / $n))"

        for ((i=0; i<8*n+n+1; i++))
        do
                echo -n "-"
                done
                echo ""

        for ((i=0; i<$n; i++))
        do
                echo -n "|   "
                echo -n "P${pid[$i]}"
                echo -n "   "
        done
        echo "|"
        for ((i=0; i<8*n+n+1; i++))
        do
                echo -n "-"
                done
                echo ""
        echo -n "0       "
        for ((i=0; i<$n; i++))
        do
                echo -n "`expr ${arrival_time[$i]} + ${tat[$i]}`"
                echo -n "           "
        done
        echo ""
}
```

GLA University, Mathura

```
echo -n "Enter the number of processes: "
read n
for ((i=0; i<$n; i++))
do
echo -n "Enter Process Id: "
read pid[$i]
echo -n "Enter arrival time: "
read arrival_time[$i]
echo -n "Enter burst time: "
read burst_time[$i]
done
findAverageTime
```

Output:

```
drpremnarya@LAPTOP-[redacted]:~$ ./fcfs.sh
Enter the number of processes: 3
Enter Process Id: 0
Enter arrival time: 1
Enter burst time: 4
Enter Process Id: 2
Enter arrival time: 2
Enter burst time: 3
Enter Process Id: 2
Enter arrival time: 3
Enter burst time: 6
----------------------------------------------------------------------------------------------------------------------
|Process Id        |Burst time          |Arrival time       |Waiting time        |Turn around time  |Completion time     |
----------------------------------------------------------------------------------------------------------------------
|0                 |4                   |1                  |0                   |4                 |5                   |
|2                 |3                   |2                  |2                   |5                 |7                   |
|2                 |6                   |3                  |4                   |10                |13                  |
----------------------------------------------------------------------------------------------------------------------
Average waiting time =2.000
Average turn around time =6.000
--------------------------
|   P0   |   P2   |   P2   |
--------------------------
0       5         7       13
```

GLA University, Mathura

2. Implement Priority CPU Scheduling Algorithm using Shell Script programming language to determine the average waiting time and average turnaround time given n processes and their burst times.

Code:

```
# Bash Script to implement Priority Scheduling Algorithm (non pre-emptive)

border(){
        z=121
        for ((i=0; i<$z; i++))
        do
           echo -n "-"
        done
        echo ""
}
arrangeArrival(){
        z=1
        for ((i=0; i<$n; i++))
        do
                for ((j=i+1; j<$n; j++))
                do
                        if [ ${arrival_time[$i]} -gt ${arrival_time[$j]} ]
                        then
                           temp=${arrival_time[$j]}
                           arrival_time[$j]=${arrival_time[$i]}
                           arrival_time[$i]=$temp

                           temp=${burst_time[$j]}
                           burst_time[$j]=${burst_time[$i]}
```

GLA University, Mathura

```
                    burst_time[$i]=$temp

                    temp=${priority[$j]}
                    priority[$j]=${priority[$i]}
                    priority[$i]=$temp

                    temp=${pid[$j]}
                    pid[$j]=${pid[$i]}
                    pid[$i]=$temp
                fi
            done
        done
}
arrangePriority(){
        z=1
        for ((i=0; i<$n; i++))
        do
            for ((j=i+1; j<$n; j++))
            do
                if [ ${arrival_time[$i]} -eq ${arrival_time[$j]} ]
                then
                    if [ ${priority[$i]} -gt ${priority[$j]} ]
                    then
                        temp=${arrival_time[$j]}
                        arrival_time[$j]=${arrival_time[$i]}
                        arrival_time[$i]=$temp
                        temp=${burst_time[$j]}
                        burst_time[$j]=${burst_time[$i]}
                        burst_time[$i]=$temp
```

GLA University, Mathura

```
                        temp=${priority[$j]}
                        priority[$j]=${priority[$i]}
                        priority[$i]=$temp

                        temp=${pid[$j]}
                        pid[$j]=${pid[$i]}
                        pid[$i]=$temp
                    fi
                fi
            done
        done
}
findWaitingTime(){
        service_time[0]=0
        waiting_time[0]=0
        for ((i=1; i<$n; i++))
        do
            z=1
            y=`expr $i - $z`
            service_time[$i]=`expr ${service_time[$y]} + ${burst_time[$y]} `
            waiting_time[$i]=`expr ${service_time[$i]} - ${arrival_time[$i]}`
            if [ ${waiting_time[$i]} -lt 0 ]
            then
                waiting_time[$i]=0
            fi
        done
}
```

GLA University, Mathura

```
findTurnAroundTime(){
        for ((i=0; i<$n; i++))
        do
                tat[$i]=`expr ${waiting_time[$i]} + ${burst_time[$i]}`
        done
}
echo -n "Enter the number of processes: "
read n
for ((i=0; i<$n; i++))
do
echo -n "Enter Process Id: "
read pid[$i]
echo -n "Enter arrival time: "
read arrival_time[$i]
echo -n "Enter burst time: "
read burst_time[$i]
echo -n "Enter priority: "
read priority[$i]
done
arrangeArrival
arrangePriority
findWaitingTime
findTurnAroundTime
total_wt=0
total_tat=0
border
printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" "Process Id" "Burst time"
"Arrival time" "Waiting time" "Turn around time" "Completion time"
border
```

GLA University, Mathura

```
for ((i=0; i<$n; i++))
do
        total_wt=`expr $total_wt + ${waiting_time[$i]}`
        total_tat=`expr ${tat[$i]} + $total_tat`
        completion_time=`expr ${arrival_time[$i]} + ${tat[$i]}`
        printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" ${pid[$i]}
${burst_time[$i]} ${arrival_time[$i]} ${waiting_time[$i]} ${tat[$i]}
$completion_time
        #echo "${burst_time[$i]}     ${arrival_time[$i]}     ${waiting_time[$i]}
${tat[$i]}         $completion_time"
done
border
avgwt=`echo "scale=3; $total_wt / $n" | bc`
echo "Average waiting time = $avgwt"
avgtat=`echo "scale=3; $total_tat / $n" | bc`
echo "Average turn around time = $avgtat"



for ((i=0; i<8*n+n+1; i++))
do
        echo -n "-"
        done
        echo ""

for ((i=0; i<$n; i++))
do
        echo -n "|   "
        echo -n "P${pid[$i]}"
        echo -n "   "
```

GLA University, Mathura

```
done
echo "|"
for ((i=0; i<8*n+n+1; i++))
do
        echo -n "-"
        done
        echo ""
echo -n "0     "
for ((i=0; i<$n; i++))
do
        echo -n "`expr ${arrival_time[$i]} + ${tat[$i]}`      "
done
echo ""
```

Output:

```
drpremnarya@LAPTOP-:~$ ./priority.sh
Enter the number of processes: 3
Enter Process Id: 1
Enter arrival time: 0
Enter burst time: 4
Enter priority: 2
Enter Process Id: 2
Enter arrival time: 1
Enter burst time: 6
Enter priority: 1
Enter Process Id: 3
Enter arrival time: 2
Enter burst time: 6
Enter priority: 5
-----------------------------------------------------------------------------------------------------------------------
|Process Id       |Burst time         |Arrival time     |Waiting time       |Turn around time  |Completion time     |
-----------------------------------------------------------------------------------------------------------------------
|1                |4                  |0                |0                  |4                 |4                   |
|2                |6                  |1                |3                  |9                 |10                  |
|3                |6                  |2                |8                  |14                |16                  |
-----------------------------------------------------------------------------------------------------------------------
Average waiting time = 3.666
Average turn around time = 9.000
--------------------------
|   P1   |   P2   |   P3   |
--------------------------
0       4          10      16
```

University, Mathura

3. Implement Round Robin CPU Scheduling Algorithm using Shell Script programming language to determine the average waiting time and average turnaround time given n processes and their burst times.

Code:

```
# Bash script to implement Round Robin CPU scheduling algorithm.

sort(){
        for ((i = 0; i<$n; i++))
        do

           for((j = 0; j<`expr $n - $i - 1`; j++))
           do

                if [ ${arrival_time[j]} -gt ${arrival_time[$((j+1))]} ]
                then
                   # swap
                   temp=${arrival_time[j]}
                   arrival_time[$j]=${arrival_time[$((j+1))]}
                   arrival_time[$((j+1))]=$temp

                   temp=${burst_time[j]}
                   burst_time[$j]=${burst_time[$((j+1))]}
                   burst_time[$((j+1))]=$temp

                   temp=${pid[j]}
                   pid[$j]=${pid[$((j+1))]}
                   pid[$((j+1))]=$temp

                   temp=${burst_time_copy[$j]}
```

GLA University, Mathura

```
    burst_time_copy[$j]=${burst_time_copy[$i]}
    burst_time_copy[$i]=$temp

    temp=${arrival_time_copy[$j]}
    arrival_time_copy[$j]=${arrival_time_copy[$i]}
    arrival_time_copy[$i]=$temp

elif [ ${arrival_time[j]} -eq ${arrival_time[$((j+1))]} ]
then
        if [ ${pid[j]} -eq ${pid[$((j+1))]} ]
        then
            temp=${arrival_time[j]}
            arrival_time[$j]=${arrival_time[$((j+1))]}
            arrival_time[$((j+1))]=$temp

            temp=${burst_time[j]}
            burst_time[$j]=${burst_time[$((j+1))]}
            burst_time[$((j+1))]=$temp

            temp=${pid[j]}
            pid[$j]=${pid[$((j+1))]}
            pid[$((j+1))]=$temp

            temp=${burst_time_copy[$j]}
            burst_time_copy[$j]=${burst_time_copy[$i]}
            burst_time_copy[$i]=$temp

            temp=${arrival_time_copy[$j]}
            arrival_time_copy[$j]=${arrival_time_copy[$i]}
            arrival_time_copy[$i]=$temp
        fi
```

GLA University, Mathura

```
                fi
            done
        done
}

border(){
        z=121
        for ((i=0; i<$z; i++))
        do
        echo -n "-"
        done
        echo ""
}

calcWaitingtime(){
        t=0
        arrival=0
        is_completed=0

        while [ $is_completed -eq 0 ]
        do
                is_completed=1
                for ((i=0; i<$n; i++))
                do
                        chart[$t]=`expr $i + 1`
                        if [ ${burst_time[$i]} -gt 0 ]
                        then
                                is_completed=0
                                if [ ${burst_time[$i]} -gt $quantum -a ${arrival_time[$i]}
-le $arrival ]
                                then
```

GLA University, Mathura

```
                        for ((j=0; j<$quantum; j++))
                        do
                                chart[$t]=`expr $i + 1`
                                ((t++))
                        done
                        #t=`expr $t + $quantum`
                        burst_time[$i]=`expr ${burst_time[$i]} -
$quantum`
                        ((arrival++))
                else
                        if [ ${arrival_time[$i]} -le $arrival ]
                        then
                                ((arrival++))
                                for ((j=0; j<${burst_time[$i]}; j++))
                                do
                                        chart[$t]=`expr $i + 1`
                                        ((t++))
                                done
                                #t=`expr $t + ${burst_time[$i]}`
                                burst_time[$i]=0
                                completion_time[$i]=$t
                        fi
                fi
        fi
    done
    if [ $is_completed -eq 1 ]
    then
        h=$t
    fi
done
```

GLA University, Mathura

```
        for ((i=0; i<$n; i++))
        do
                tat[$i]=`expr ${completion_time[$i]} - ${arrival_time_copy[$i]}`
                waiting_time[$i]=`expr ${tat[$i]} - ${burst_time_copy[$i]}`
        done

        total_wt=0
        total_tat=0
        border
        printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" "Process Id" "Burst time"
"Arrival time" "Waiting time" "Turn around time" "Completion time"
        border
        for ((i=0; i<$n; i++))
        do
                total_wt=`expr $total_wt + ${waiting_time[$i]}`
                total_tat=`expr ${tat[$i]} + $total_tat`
                completion_time=`expr ${arrival_time[$i]} + ${tat[$i]}`
                printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" ${pid[$i]}
${burst_time_copy[$i]} ${arrival_time_copy[$i]} ${waiting_time[$i]} ${tat[$i]}
$completion_time
                #echo "${burst_time[$i]}     ${arrival_time[$i]}     ${waiting_time[$i]}
${tat[$i]}       ${completion_time[$i]}"
        done
        border
        avgwt=`echo "scale=3; $total_wt / $n" | bc`
        echo "Average waiting time = $avgwt"
        avgtat=`echo "scale=3; $total_tat / $n" | bc`
        echo "Average turn around time = $avgtat"


        count_cols=1
```

GLA University, Mathura

```
cols_id[0]=${chart[0]}
cols[0]=0
j=1
for ((i=1; i<$h; i++))
do
        if [ ${chart[$i]} -ne ${chart[`expr $i - 1`]} ]
        then
                ((count_cols++))
                cols[$j]=$i
                cols_id[$j]=${chart[$i]}
                ((j++))
        fi
done

echo ""

for ((i=0; i<8*count_cols+count_cols+1; i++))
do
        echo -n "-"
        done
        echo ""

for ((i=0; i<$count_cols; i++))
do
        echo -n "|   "
        echo -n "P${cols_id[$i]}"
        echo -n "   "
done
echo "|"
for ((i=0; i<8*count_cols+count_cols+1; i++))
do
```

GLA University, Mathura

```
                echo -n "-"
                done
                echo ""
        echo -n "0      "
        for ((i=1; i<$count_cols; i++))
        do
                echo -n "${cols[$i]}"
                echo -n "          "
        done
        echo -n "$h"
        echo ""
}

echo -n "Enter the number of processes: "
read n
for ((i=0; i<$n; i++))
do
echo -n "Enter Process Id: "
read pid[$i]
echo -n "Enter arrival time: "
read arrival_time[$i]
arrival_time_copy[$i]=${arrival_time[$i]}
echo -n "Enter burst time: "
read burst_time[$i]
burst_time_copy[$i]=${burst_time[$i]}
done
echo -n "Enter quantum size: "
read quantum
sort
calcWaitingtime
```

GLA University, Mathura

Output:

```
drpremnarya@LAPTOP-[redacted]:~$ ./round_robin.sh
Enter the number of processes: 3
Enter Process Id: 1
Enter arrival time: 0
Enter burst time: 4
Enter Process Id: 2
Enter arrival time: 2
Enter burst time: 6
Enter Process Id: 3
Enter arrival time: 2
Enter burst time: 3
Enter quantum size: 3
-----------------------------------------------------------------------------------------------------------------
|Process Id        |Burst time          |Arrival time      |Waiting time        |Turn around time  |Completion time     |
-----------------------------------------------------------------------------------------------------------------
|1                 |4                   |0                 |0                   |4                 |4                   |
|2                 |6                   |2                 |5                   |11                |13                  |
|3                 |3                   |2                 |5                   |8                 |10                  |
-----------------------------------------------------------------------------------------------------------------
Average waiting time = 3.333
Average turn around time = 7.666

-----------------------------------
|   P1   |   P2   |   P3   |   P2   |
-----------------------------------
0        4        7        10       13
```

GLA University

4. Implement Shortest Job First (Non-Pre-emptive) CPU Scheduling Algorithm using Shell Script programming language to determine the average waiting time and average turnaround time given n processes and their burst times.

Code:

```
# Bash Script to implement Shortest Job First Scheduling Algorithm (non pre-emptive)

border(){
        z=121
        for ((i=0; i<$z; i++))
        do
        echo -n "-"
        done
        echo ""
}

arrangeArrival(){
        z=1
        for ((i=0; i<$n; i++))
        do
                for ((j=i+1; j<$n; j++))
                do
                        if [ ${arrival_time[$i]} -gt ${arrival_time[$j]} ]
                        then
                           temp=${arrival_time[$j]}
                           arrival_time[$j]=${arrival_time[$i]}
                           arrival_time[$i]=$temp

                           temp=${burst_time[$j]}
                           burst_time[$j]=${burst_time[$i]}
                           burst_time[$i]=$temp
```

GLA University, Mathura

```
                temp=${pid[$j]}
                pid[$j]=${pid[$i]}
                pid[$i]=$temp
            fi
        done
    done
}

arrangeBurst(){
    z=1
    for ((i=0; i<$n; i++))
    do
        for ((j=i+1; j<$n; j++))
        do
            if [ ${arrival_time[$i]} -eq ${arrival_time[$j]} ]
            then
                if [ ${burst_time[$i]} -gt ${burst_time[$j]} ]
                then
                    temp=${arrival_time[$j]}
                    arrival_time[$j]=${arrival_time[$i]}
                    arrival_time[$i]=$temp

                    temp=${burst_time[$j]}
                    burst_time[$j]=${burst_time[$i]}
                    burst_time[$i]=$temp

                    temp=${pid[$j]}
                    pid[$j]=${pid[$i]}
                    pid[$i]=$temp
                fi
            fi
        done
    done
}
```

GLA University, Mathura

```
completionTime(){
        completion_time[0]=`expr ${arrival_time[0]} + ${burst_time[0]}`
        tat[0]=`expr ${completion_time[0]} - ${arrival_time[0]}`
        waiting_time[0]=`expr ${tat[0]} - ${burst_time[0]}`
        for ((i=1; i<$n; i++))
        do
                temp=${completion_time[`expr $i - 1`]}
                low=${burst_time[$i]}
                for ((j=i; j<$n; j++))
                do
                        if [ $temp -ge ${arrival_time[$j]} ]
                        then
                                if [ $low -ge ${burst_time[$j]} ]
                                then
                                        low=${burst_time[$j]}
                                        val=$j
                                fi
                        fi
                done
                completion_time[$val]=`expr $temp + ${burst_time[$val]}`
                tat[$val]=`expr ${completion_time[$val]} - ${arrival_time[$val]}`
                waiting_time[$val]=`expr ${tat[$val]} - ${burst_time[$val]}`

                if [ $val -ne $i ]
                then
                        temp=${arrival_time[$val]}
                        arrival_time[$val]=${arrival_time[$i]}
                        arrival_time[$i]=$temp

                        temp=${burst_time[$val]}
                        burst_time[$val]=${burst_time[$i]}
                        burst_time[$i]=$temp
```

GLA University, Mathura

```
                    temp=${pid[$val]}
                    pid[$val]=${pid[$i]}
                    pid[$i]=$temp

                    temp=${completion_time[$val]}
                    completion_time[$val]=${completion_time[$i]}
                    completion_time[$i]=$temp

                    temp=${waiting_time[$val]}
                    waiting_time[$val]=${waiting_time[$i]}
                    waiting_time[$i]=$temp

                    temp=${tat[$val]}
                    tat[$val]=${pid[$i]}
                    tat[$i]=$temp
                fi
        done
}

echo -n "Enter the number of processes: "
read n
for ((i=0; i<$n; i++))
do
echo -n "Enter Process Id: "
read pid[$i]
echo -n "Enter arrival time: "
read arrival_time[$i]
echo -n "Enter burst time: "
read burst_time[$i]
done
arrangeArrival
arrangeBurst
completionTime
total_wt=0
```

GLA University, Mathura

```
total_tat=0
border
printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" "Process Id" "Burst time" "Arrival time"
"Waiting time" "Turn around time" "Completion time"
border
for ((i=0; i<$n; i++))
do
        total_wt=`expr $total_wt + ${waiting_time[$i]}`
        total_tat=`expr ${tat[$i]} + $total_tat`
        completion_time=`expr ${arrival_time[$i]} + ${tat[$i]}`
        printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" ${pid[$i]} ${burst_time[$i]}
${arrival_time[$i]} ${waiting_time[$i]} ${tat[$i]} $completion_time
        #echo "${burst_time[$i]}     ${arrival_time[$i]}     ${waiting_time[$i]}       ${tat[$i]}
$completion_time"
done
border
avgwt=`echo "scale=3; $total_wt / $n" | bc`
echo "Average waiting time = $avgwt"
avgtat=`echo "scale=3; $total_tat / $n" | bc`
echo "Average turn around time = $avgtat"



for ((i=0; i<8*n+n+1; i++))
do
        echo -n "-"
        done
        echo ""

for ((i=0; i<$n; i++))
do
        echo -n "|   "
        echo -n "P${pid[$i]}"
        echo -n "   "
done
```

GLA University, Mathura

```
echo "|"
for ((i=0; i<8*n+n+1; i++))
do
        echo -n "-"
        done
        echo ""
echo -n "0       "
for ((i=0; i<$n; i++))
do
        echo -n "`expr ${arrival_time[$i]} + ${tat[$i]}`          "
done
echo ""
```

Output:

```
drpremnarya@LAPTOP-      :~$ ./sjf_non_preemptive.sh
Enter the number of processes: 3
Enter Process Id: 1
Enter arrival time: 0
Enter burst time: 4
Enter Process Id: 2
Enter arrival time: 1
Enter burst time: 3
Enter Process Id: 3
Enter arrival time: 2
Enter burst time: 5
-----------------------------------------------------------------------------------------------------------------
|Process Id       |Burst time         |Arrival time     |Waiting time       |Turn around time  |Completion time    |
-----------------------------------------------------------------------------------------------------------------
|1                |4                  |0                |0                  |4                 |4                  |
|2                |3                  |1                |3                  |6                 |7                  |
|3                |5                  |2                |5                  |10                |12                 |
-----------------------------------------------------------------------------------------------------------------
Average waiting time = 2.666
Average turn around time = 6.666
--------------------------
|   P1   |   P2   |   P3   |
--------------------------
0       4       7       12
```

5. Implement SJF Pre-emptive (or Shortest Remaining Time First (SRTF)) CPU Scheduling Algorithm using Shell Script programming language to determine the average waiting time and average turnaround time given n processes and their burst times.

Code:

```
# Bash script to implement Shortest Job First Scheduling Algorithm (Pre-emptive)

border(){
        z=121
        for ((i=0; i<$z; i++))
        do
        echo -n "-"
        done
        echo ""
}

arrangeArrival(){
        z=1
        for ((i=0; i<$n; i++))
        do
                for ((j=i+1; j<$n; j++))
                do
                        if [ ${arrival_time[$i]} -gt ${arrival_time[$j]} ]
                        then
                           temp=${arrival_time[$j]}
                           arrival_time[$j]=${arrival_time[$i]}
                           arrival_time[$i]=$temp

                           temp=${burst_time[$j]}
                           burst_time[$j]=${burst_time[$i]}
                           burst_time[$i]=$temp
```

GLA University, Mathura

```
                        temp=${burst_time_copy[$j]}
                        burst_time_copy[$j]=${burst_time_copy[$i]}
                        burst_time_copy[$i]=$temp

                        temp=${pid[$j]}
                        pid[$j]=${pid[$i]}
                        pid[$i]=$temp
                    fi
                done
        done
}

arrangeBurst(){
        z=1
        for ((i=0; i<$n; i++))
        do
                for ((j=i+1; j<$n; j++))
                do
                    if [ ${arrival_time[$i]} -eq ${arrival_time[$j]} ]
                    then
                        if [ ${burst_time[$i]} -gt ${burst_time[$j]} ]
                        then
                            temp=${arrival_time[$j]}
                            arrival_time[$j]=${arrival_time[$i]}
                            arrival_time[$i]=$temp

                            temp=${burst_time[$j]}
                            burst_time[$j]=${burst_time[$i]}
                            burst_time[$i]=$temp

                            temp=${burst_time_copy[$j]}
                            burst_time_copy[$j]=${burst_time_copy[$i]}
                            burst_time_copy[$i]=$temp
```

GLA University, Mathura

```
                        temp=${pid[$j]}
                        pid[$j]=${pid[$i]}
                        pid[$i]=$temp
                    fi
                fi
            done
        done
}

timecalc(){
        is_completed=0
        current_time=0
        cp=0
        count=0
        max=1000
        for ((i=0; i<$n; i++))
        do
            if [ ${arrival_time[$i]} -le $current_time ]
            then
                if [ ${burst_time[$i]} -lt $max ]
                then
                    if [ ${burst_time[$i]} -ne 0 ]
                    then
                        cp=$i
                        max=${burst_time[$i]}
                        if [ ${burst_time[$i]} -eq ${burst_time_copy[$i]} ]
                        then
                            waiting_time[$i]=$current_time
                        fi
                    fi
                fi
            fi
        done
```

GLA University, Mathura

```
while [ $is_completed -eq 0 ]
do
        if [ $count -eq $n ]
        then
                is_completed=1
                h=$current_time
        fi
        chart[$current_time]=`expr $cp + 1`
        ((current_time++))
        if [ ${burst_time[$cp]} -gt 0 ]
        then
                burst_time[$cp]=`expr ${burst_time[$cp]} - 1`
                max=${burst_time[$cp]}
                if [ ${burst_time[$cp]} -eq 0 ]
                then
                        ((count++))
                        completion_time[$cp]=$current_time
                        max=1000
                fi
        fi
        prevcp=$cp
        for ((i=0; i<$n; i++))
        do
                if [ ${arrival_time[$i]} -le $current_time ]
                then
                        if [ ${burst_time[$i]} -lt $max ]
                        then
                                if [ ${burst_time[$i]} -ne 0 ]
                                then
                                        cp=$i
                                        max=${burst_time[$i]}
                                fi
                        fi
                fi
```

GLA University, Mathura

```
                done
                if [ $prevcp -ne $cp ]
                then
                        waiting_time[$i]=$current_time
                fi

        done
        for ((i=0; i<$n; i++))
        do
                waiting_time[$i]=`expr ${completion_time[$i]} - ${arrival_time[$i]} -
${burst_time_copy[$i]}`
                if [ ${waiting_time[$i]} -lt 0 ]
                then
                        waiting_time[$i]=0
                fi
                tat[$i]=`expr ${waiting_time[$i]} + ${burst_time_copy[$i]}`
        done
        total_wt=0
        total_tat=0
        border
        printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" "Process Id" "Burst time"
"Arrival time" "Waiting time" "Turn around time" "Completion time"
        border
        for ((i=0; i<$n; i++))
        do
                total_wt=`expr $total_wt + ${waiting_time[$i]}`
                total_tat=`expr ${tat[$i]} + $total_tat`
                completion_time=`expr ${arrival_time[$i]} + ${tat[$i]}`
                printf "|%-18s|%-20s|%-18s|%-20s|%-18s|%-20s|\n" ${pid[$i]}
${burst_time_copy[$i]} ${arrival_time[$i]} ${waiting_time[$i]} ${tat[$i]}
$completion_time
                #echo "${burst_time[$i]}     ${arrival_time[$i]}     ${waiting_time[$i]}
${tat[$i]}        ${completion_time[$i]}"
        done
```

GLA University, Mathura

```
border
avgwt=`echo "scale=3; $total_wt / $n" | bc`
echo "Average waiting time = $avgwt"
avgtat=`echo "scale=3; $total_tat / $n" | bc`
echo "Average turn around time = $avgtat"

count_cols=1
cols_id[0]=${chart[0]}
cols[0]=0
j=1
for ((i=1; i<$h; i++))
do
        if [ ${chart[$i]} -ne ${chart[`expr $i - 1`]} ]
        then
                ((count_cols++))
                cols[$j]=$i
                cols_id[$j]=${chart[$i]}
                ((j++))
        fi
done

echo ""

for ((i=0; i<8*count_cols+count_cols+1; i++))
do
        echo -n "-"
        done
        echo ""

for ((i=0; i<$count_cols; i++))
do
        echo -n "|   "
        echo -n "P${cols_id[$i]}"
        echo -n "   "
```

GLA University, Mathura

```
        done
        echo "|"
        for ((i=0; i<8*count_cols+count_cols+1; i++))
        do
                echo -n "-"
                done
                echo ""
        echo -n "0       "
        for ((i=1; i<$count_cols; i++))
        do
                echo -n "${cols[$i]}"
                echo -n "            "
        done
        echo -n "$h"
        echo ""
}

echo -n "Enter the number of processes: "
read n
for ((i=0; i<$n; i++))
do
echo -n "Enter Process Id: "
read pid[$i]
echo -n "Enter arrival time: "
read arrival_time[$i]
echo -n "Enter burst time: "
read burst_time[$i]
burst_time_copy[$i]=${burst_time[$i]}
done
arrangeArrival
arrangeBurst
timecalc
```

GLA University, Mathura

Output:

```
drpremnarya@LAPTOP-      :~$ ./sjf_preemptive.sh
Enter the number of processes: 3
Enter Process Id: 1
Enter arrival time: 1
Enter burst time: 3
Enter Process Id: 2
Enter arrival time: 1
Enter burst time: 4
Enter Process Id: 3
Enter arrival time: 3
Enter burst time: 5
-----------------------------------------------------------------------------------------------------------------------
|Process Id        |Burst time          |Arrival time       |Waiting time        |Turn around time  |Completion time     |
-----------------------------------------------------------------------------------------------------------------------
|1                 |3                   |1                  |0                   |3                 |4                   |
|2                 |4                   |1                  |2                   |6                 |7                   |
|3                 |5                   |3                  |4                   |9                 |12                  |
-----------------------------------------------------------------------------------------------------------------------
Average waiting time = 2.000
Average turn around time = 6.000

--------------------------
|   P1   |   P2   |   P3   |
--------------------------
```

GLA University, M

Thank You

GLA University, Mathura